National University

of Computer and Emerging Sciences
Chiniot-Faisalabad Campus

CS 4059 – Fundamentals of Computer Vision

Assignment Number 1

Introduction to Computer Vision

Spring 2026

Maximum Marks: 100 **Due Date:** 3 March 2026

Submitted By

Name: Nouman Shabbir

Student ID: 22F-3788

Section: 8A

Submitted To

Engr. Mubeen Tariq
Lecturer – CE Department

Submission Date

3 March 2026

GitHub Link: https://github.com/Nouman14-ai/Computer_Vision_Basics

Details of Obtained Marks

Question Number	1	2	3	**Total**
Total Marks	40	30	30	**100**
Obtained Marks				

Contents

Introduction 3
Question 1: Image Enhancement Pipeline 3
Part (a) Histogram Equalization 3
Part (b) Contrast Stretching 3
Part (c) Gamma (Power-Law) Transformation 4
Question 2: Image Filtering and Convolution 4
Part (a) 2D Convolution 4
Part (b) Gaussian Filter 5
Part (c) Separable Filtering 5
Question 3: Edge Detection and Gradient Computation 5
Part (a) Image Gradients 5
Part (b) Canny Edge Detection 5
Part (c) Edge Detection Comparison 6
Challenges & Solutions 6
Conclusion 6

Introduction

This assignment implements fundamental computer vision algorithms from scratch, including **image enhancement, filtering, convolution, and edge detection**.
The objectives are:

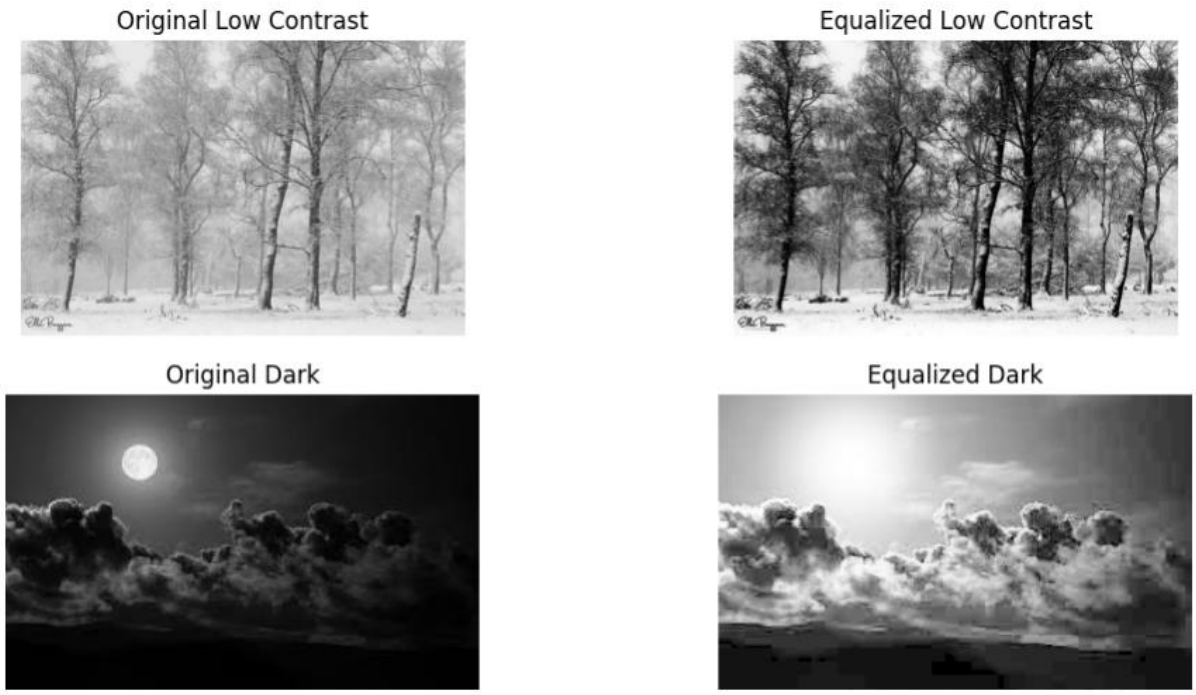
- Understand and implement **histogram equalization, contrast stretching, and gamma transformation**.
- Implement **2D convolution, Gaussian filtering, and separable filtering optimization**.
- Compute **image gradients**, implement **Canny edge detection**, and compare multiple edge detection methods.
- Learn to evaluate results and compare custom implementations with library functions like OpenCV.

All code is written primarily using **NumPy and Matplotlib**. OpenCV is used only for result comparison.

Question 1: Image Enhancement Pipeline

Part (a) Histogram Equalization

Explanation:
Histogram equalization enhances image contrast by spreading out the intensity distribution. It calculates the **cumulative distribution function (CDF)** of pixel intensities and maps original pixels to new values.



Observations:

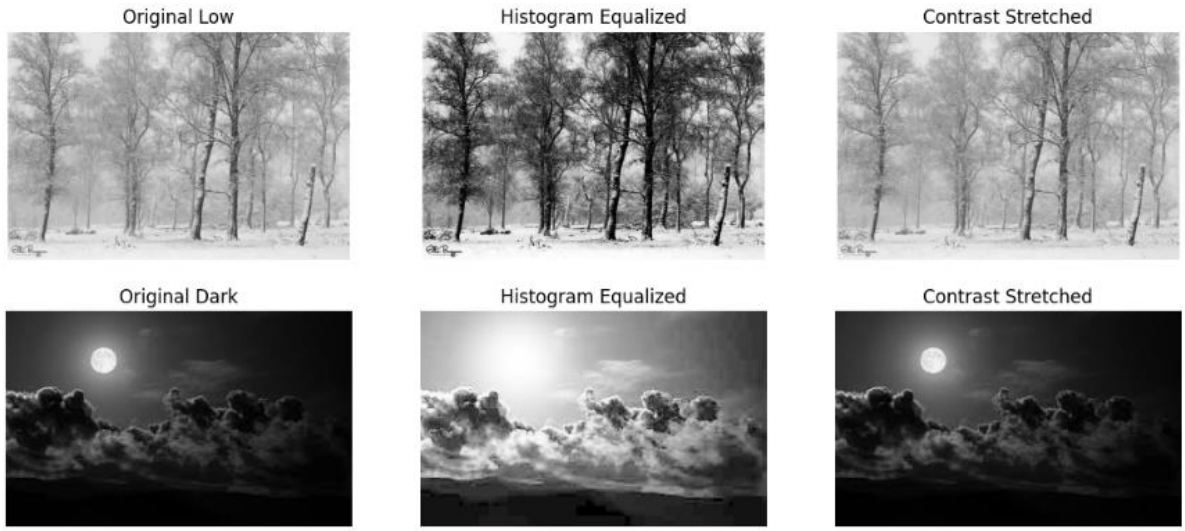
- Equalization improves visibility of dark or low-contrast regions.
- OpenCV results are almost identical to the custom implementation.

Part (b) Contrast Stretching

Explanation:
Contrast stretching maps pixel values linearly using:

$$s = \frac{(r - r_{\min})}{(r_{\max} - r_{\min})} \times 255$$

It expands the intensity range to cover the full 0–255 scale.



Observations:

- Contrast stretching is effective for underexposed images.
- Histogram equalization sometimes introduces noise in uniform regions.

Part (c) Gamma (Power-Law) Transformation

Explanation:

Gamma correction adjusts brightness non-linearly:

$$s = c \cdot r^{\gamma}$$

1. $\gamma < 1 \rightarrow$ image brightens
2. $\gamma > 1 \rightarrow$ image darkens
3. $\gamma = 1 \rightarrow$ image unchanged

Observations:

- Useful for correcting underexposed or overexposed images.

Question 2: Image Filtering and Convolution

Part (a) 2D Convolution

Explanation:

- Implemented convolution from scratch using NumPy.
- Three padding strategies: zero, replicate, no padding.
- I tested with box filter, identity kernel, and edge detection kernel.

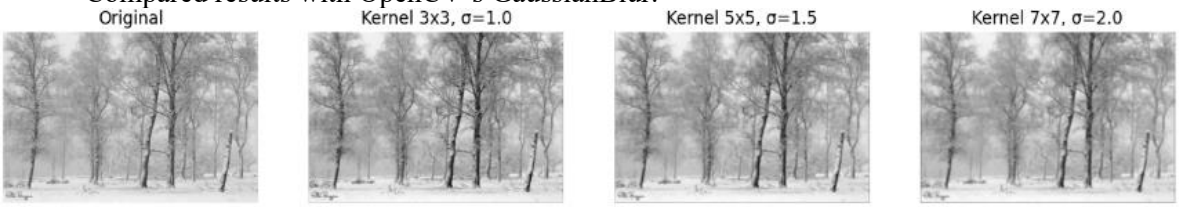
Observations:

- Edge kernels highlight edges effectively.
- Zero-padding avoids index errors, replicate padding preserves edges at borders.

Part (b) Gaussian Filter

Explanation:

- Generated Gaussian kernel with different sizes and σ values.
- Applied convolution to smooth images.
- Compared results with OpenCV's GaussianBlur.



Observations:

- Larger kernel size or σ produces stronger blur.
- Gaussian preserves edges better than a box filter.

Part (c) Separable Filtering

Explanation:

- Gaussian kernel separated into 1D horizontal and vertical filters.
- Reduced computation compared to 2D convolution.

Observations:

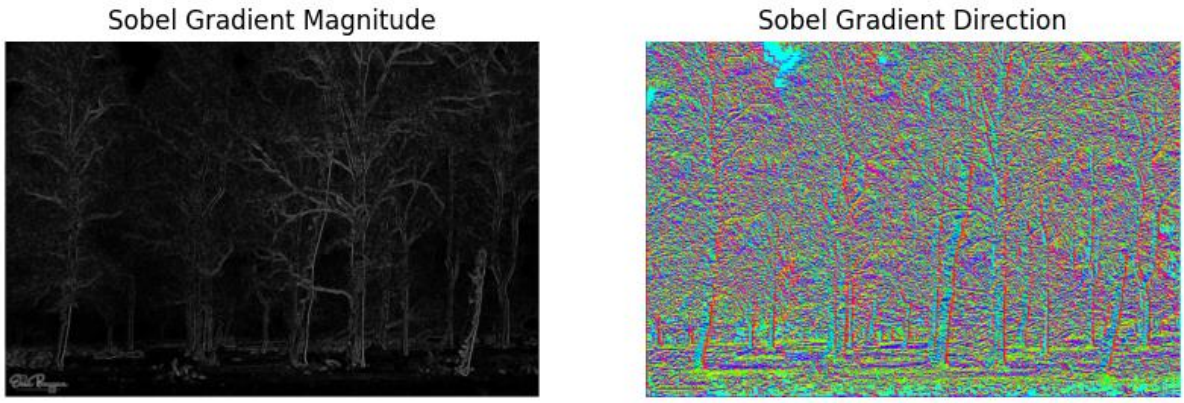
- Execution time for separable filtering is significantly lower.
- Output visually identical to 2D convolution.

Question 3: Edge Detection and Gradient Computation

Part (a) Image Gradients

Explanation:

- Implemented gradient computation using Central, Sobel, Prewitt, and Roberts kernels.
- Computed horizontal (Gx), vertical (Gy) gradients, magnitude, and direction.



Observations:

- Sobel and Prewitt give smoother results; Roberts detects fine edges but is sensitive to noise.

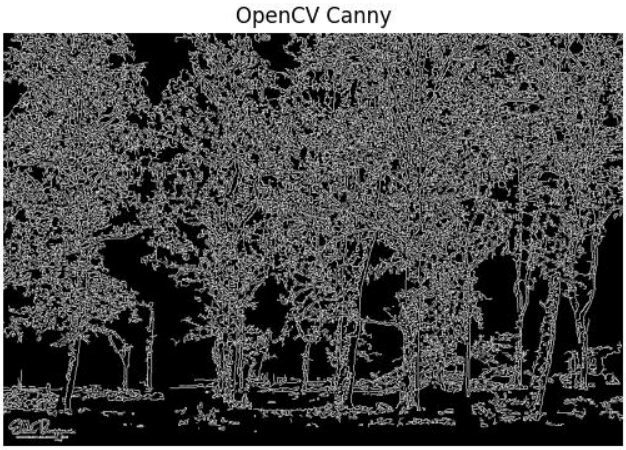
Part (b) Canny Edge Detection

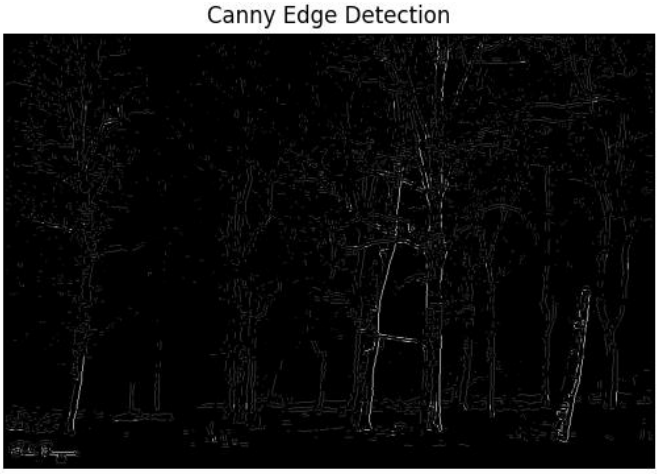
Explanation:

Steps:

- Gaussian smoothing
- Gradient computation

- Non-maximum suppression
- Hysteresis thresholding





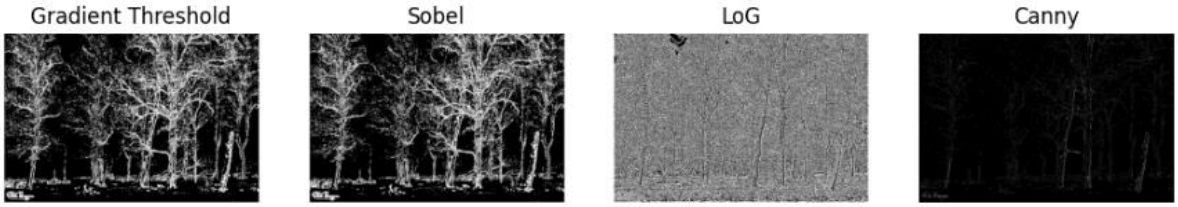
Observations:

- Proper threshold selection is crucial.
- Custom implementation matches OpenCV closely.

Part (c) Edge Detection Comparison

Explanation:

- Compared Gradient thresholding, Sobel thresholding, Laplacian of Gaussian (LoG), and Canny.
- Tested robustness with added Gaussian noise.



Observations:

- Canny is the most robust to noise and accurately detects edges.
- LoG detects edges but produces more false positives in noisy regions.

Challenges & Solutions

- Implementing non-maximum suppression correctly
- Handling image borders in convolution
- GitHub save issues → solved by manual upload

Conclusion

- Successfully implemented core computer vision algorithms from scratch.

- Observed differences between library and custom implementations.
- Learned practical application of image enhancement, filtering, and edge detection.